Unified Training–Learning–Inference–Reinforcement Dynamics Across Neural Systems

Mian Wang

Wuhan Polytechnic University, Wuhan, China

Corresponding author. Email: wangmian945@gmail.com

Although training, learning, inference, and reinforcement have been extensively studied, their distinct dynamical roles and connections have lacked a unified experimental characterization. Nine nanoGPT experiments established training–learning–inference relations: realized updates elicit state-dependent finite-amplitude responses, learning reorganizes distributed functional support across capability boundaries, and inference recruits that support without changing the learned state. Factors identified in the fifth experiment yielded a direct predictor of post-update capability transitions, achieving 91.43% accuracy and 91.49% four-way macro recall across 5,088 targets. We tested these relations and prediction across five additional neural architectures. In a seventh system, concentrated correct feedback eroded previously mastered skills; rebalanced feedback restored their accuracy from 70.31% to 99.48%. Linking inference consequences to subsequent training closes these relations into unified training–learning–inference–reinforcement dynamics.

Training, generalization, capability formation, and inference are central problems in neural-network research. Different fields have developed influential theories to study them.

In *optimization theory*, the Neural Tangent Kernel (NTK) describes parameter-induced output changes through the network Jacobian and the corresponding kernel in specified training regimes (*1*). Edge of Stability studies training behavior in relation to the largest Hessian eigenvalue, learning rate, and loss trajectory (*2*). Sharpness-Aware Minimization (SAM) constructs training updates using loss changes in a parameter neighborhood to improve generalization (*3*). Here, the *current*

state is represented by parameters and their local derivatives or curvature; *realized updates* are produced by the optimizer; and *responses* appear as changes in model outputs or loss.

In *representation learning and generalization*, Spectral Bias examines frequency-dependent learning rates of functions or kernel modes (*4*). Implicit Bias studies why an optimizer selects particular solutions, such as maximum-margin directions on separable data (*5*). Neural Collapse measures class-feature means, within-class variability, and classifier geometry, revealing continuing representational change after training accuracy saturates (*6*). Information Bottleneck theory studies mutual information among inputs, hidden representations, and targets, including the retention and compression of task-relevant information (*7*). Their *internal organizations* take the form of spectral modes, optimizer-selected solutions, feature geometry, or information structure; their *functional changes* appear in learning rates, evolving solutions, and predictive performance.

In *capability formation*, Grokking captures the long delay between fitting training examples and achieving strong test generalization (*8*). Subsequent circuit studies track the formation and reorganization of internal computations during this delay (*9*). Double Descent studies how test error can rise near interpolation and fall again as model size, training time, or data conditions change (*10*, *11*). These studies measure the *current state* through training phase or interpolation regime, *internal change* through circuit and solution evolution, and *observable capability* through accuracy and generalization error.

In *continual learning*, catastrophic forgetting occurs when training on a new task reduces performance on earlier tasks. Elastic Weight Consolidation (EWC) estimates parameter importance using Fisher information from earlier tasks and penalizes changes to important parameters (*12*). Gradient Episodic Memory (GEM) constrains new-task gradients using their inner products with previous-task gradients (*13*). Orthogonal Gradient Descent (OGD) projects new gradients into directions that approximately preserve earlier-task outputs (*14*). In these methods, the *existing state* includes previously learned parameters and function-sensitive directions; *updates* are constrained new-task actions; *interactions* occur through overlapping gradient or functional subspaces; and *capability changes* are measured as retention, forgetting, or transfer.

In *inference research*, Attention uses queries, keys, and values to compute input-dependent information weights (*15*). In-context learning studies how examples and task information provided in the context affect predictions while model parameters remain fixed (*16*). Chain-of-Thought

prompting studies how intermediate reasoning steps affect performance on complex tasks (*17*). The *current state* is the trained model; *input conditions* include the query, context, and intermediate computations; *internal computation* varies with attention and activations; and *functional performance* is observed in the resulting predictions.

In *reinforcement and feedback-driven training*, reinforcement learning from human feedback (RLHF) uses preference and reward signals to adjust model behavior (*18*). Self-training creates further training data from a model's predictions (*19*). Recursive model-collapse studies examine the loss of low-probability modes when successive models are trained on data generated by their predecessors (*20*). Here, *feedback* is supplied through rewards, preferences, or generated-data distributions; *training actions* are the resulting updates; and subsequent *functional change* is observed through behavior, accuracy, and generated distributions.

These studies use different mathematical objects—kernels, Hessians, gradients, class features, mutual information, task scores, attention weights, and reward signals—yet six recurring categories can be identified: *current state, realized update, functional response, internal organization, observable capability, and feedback. The core TLI relations were established experimentally before this cross-field comparison; identifying these recurring variables here is a way to help readers understand the theory, not the process by which it was discovered.*

Figure 1 illustrates the four linked dynamical roles.

Experimental Establishment of TLI Open-Loop Dynamics

The nine nanoGPT experiments that established TLI involved a substantially broader set of native interventions, controlled comparisons, and independent validations than can be presented within the main text. This section summarizes their key experimental findings and the resulting theoretical relations. Complete protocols, execution records, negative results, and independent validators are provided in the accompanying repository.

Experimental Instruments and Evidence Control

These experiments required instruments that were developed as the measurement questions arose, rather than assumed before the theory. Their common evidence substrate was the Generation-

Fact Graph (GFG), which links recorded model and optimizer states, actual update occurrences, interventions, outputs, and their source identities. For Training, realized-update causal forks restored matched receiving states and compared full, skipped, parameter-only, optimizer-memory, and reciprocal transfers, separating an update from the state on which it acts. A finite-amplitude path instrument then applied the same recorded update at frozen amplitude values to measure nonlinear numerical responses and discrete transitions. For Learning, CSRG-4C used repeated baseline, single-component, and paired-component gates to measure necessity, backup, substitution, synergy, and failure slack in the functional support of registered target groups. Finally, an identity-aligned boundary ledger joined each registered target to the exact pre-update state and realized update, then separated pre-response records from post-update adjudication of correctness. The instruments enabled the subsequent experimental tests; frozen protocols, negative controls, and independent validators constrained their interpretation.

Training: Realized Updates and Receiving States

At a registered training step, the parameter–optimizer state is $Z_t = (\theta_t, \omega_t)$, and the realized parameter update is $\Delta\theta_t = \theta_{t+1} - \theta_t$.

TL-E01 revealed capability formation, decline, and recovery that could not be fully characterized by loss, accuracy, or training step alone. TL-E02 further showed that pausing effective parameter–optimizer evolution delays capability formation, while the clipping-threshold control did not reproduce this effect.

TL-E03 used full/skip branches and reciprocal update transplants to establish that the same realized update produces different functional responses in different receiving states:

$$R_q(\theta_A, \Delta\theta_B) \quad \text{vs.} \quad R_q(\theta_B, \Delta\theta_B), \tag{1}$$

where $R_q(\theta, \Delta) = m_q(\theta + \Delta) - m_q(\theta)$. Separate optimizer-memory exchanges examined how historical optimizer states affect update formation:

$$\Delta^{(a,b)} = \mathrm{OptStep}(\theta_t, g_t, \omega_{a,b}). \tag{2}$$

TL-E04 measured the complete finite-amplitude response to an actual update:

$$r_{t,q}(\alpha) = m_q(\theta_t + \alpha\Delta\theta_t) - m_q(\theta_t). \tag{3}$$

The experiments identified near-linear, saturating, accelerating, turnback, and sign-reversal responses. TL-E05 subsequently identified three primary response-conditioning families: target-boundary state (F1), target-specific update geometry (F3), and parameter–optimizer receiving state (F5).

Learning: Distributed Support and Capability Boundaries

TL-E06 used reversible component interventions to establish distributed functional support, including component necessity, backup, substitution, synergy, and failure slack.

TL-E07 then examined how realized training updates persistently reorganize this support:

$$\Delta P_{t,q} = P_q(\theta_t + \Delta\theta_t) - P_q(\theta_t). \tag{4}$$

Across 72 experimental sections, 1,636 valid support-allocation transitions and 228 primary-support hand-offs were recorded.

TL-E08 linked these internal changes to observable capability through identity-aligned target readout boundaries. The signs of the pre- and post-update margins distinguish maintenance, loss, non-acquisition, and acquisition or recovery. Learning is thus identified with persistent functional reorganization, while capability transitions are determined by crossings of target-specific readout boundaries.

Inference: Frozen Recruitment of Learned Support

INF-E01 examined 13 nanoGPT training histories and 52 checkpoints. With persistent learned states frozen, the experiment measured query-conditioned component effects, non-additive support interactions, and changes produced by replacing trained components with their exact pre-formation versions.

At a fixed learned state L^*, inference recruits and combines learned support:

$$S_{q,c}^{\text{active}} = \Pi_{q,c}(L^*), \qquad y_{q,c} = H_{q,c}(S_{q,c}^{\text{active}}). \tag{5}$$

Non-additive component interactions are measured by

$$I_{ij,q} = f_{\text{full}} - f_{-i} - f_{-j} + f_{-i,-j}, \tag{6}$$

while learned-version dependence is tested through

$$D_{j,q} = f(q;\theta^{*}) - f(q;\theta^{(j\leftarrow \mathrm{pre})}). \tag{7}$$

Exact restoration verified that these interventions did not persistently alter the learned state. The experiments establish Inference as the query-conditioned recruitment and combination of support formed through Learning.

Together, these nine experiments established the connected Training–Learning–Inference relations, forming the TLI open-loop dynamics. Figure 2 summarizes the experimental evidence across all four TLIR roles.

Prediction and Cross-System Validation

Theory-Derived Target-Boundary Prediction

The response-factor analysis in TL-E05 identified three primary conditioning factors: the current target-boundary state (F1), target-specific update geometry (F3), and parameter–optimizer receiving state (F5). These findings led directly to TL-P01, a predictor of immediate target-boundary outcomes following an actual formed update.

For target y, competitor c, and hidden representation h, let $b_c = w_y - w_c$ and $\Delta b_c = \Delta w_y - \Delta w_c$. Using the first and second directional derivatives $h' = Dh[\Delta\theta_t]$ and $h'' = D^2h[\Delta\theta_t, \Delta\theta_t]$, the predictor computes

$$\begin{aligned}\widehat{g}_{y,c} = {} & b_c^{\top}h + \Delta b_c^{\top}h + b_c^{\top}h' \\ & + \Delta b_c^{\top}h' + \tfrac{1}{2}b_c^{\top}h''.\end{aligned} \tag{8}$$

The target is predicted correct if $\min_{c\neq y}\widehat{g}_{y,c} > 0$. Combining predicted post-update correctness with the known pre-update state gives four capability transitions. Predictions use the already formed update and pre-response information, without reading actual post-update target outputs.

Cross-System Validation

We subsequently tested the established TLI relations and their predictive consequences in five additional neural systems. Table 1 summarizes the seven systems used in the study, including the separate GRU reinforcement experiment examined in the reinforcement section.

Each of the five additional TLI systems was examined through realized-update interventions, receiving-state comparisons, finite-amplitude responses, component-coalition support measurements, target-boundary outcomes, and frozen-inference tests. Cross-system implementations evaluated all 16 coalitions of four registered components, with exact Shapley effects:

$$\phi_{j,q} = \sum_{C \subseteq \mathscr{C} \setminus \{j\}} \frac{|C|!(3-|C|)!}{4!} \left[v_q(C \cup \{j\}) - v_q(C) \right]. \tag{9}$$

Here $v_q(C)$ denotes the target margin under active component coalition C.

All 42 ResNet, 18 diffusion U-Net, and 36 Vision-MoE parameter receiving-state exchanges exceeded their registered response-difference thresholds. Finite-amplitude nonlinear responses occurred in 32.23%, 19.25%, and 35.71% of their respective target-update records, while support reallocation exceeded its specified threshold in 91.41%, 97.02%, and 80.43%. Their frozen-inference experiments further confirmed query-conditioned support, non-additive interactions, and dependence on exact trained component versions.

The GNN reproduced the main TLI relations across three seeds, although one seed failed the registered nonlinear-response frequency criterion (1.56%). The hard-spiking SNN passed all twelve registered TLI criteria across four seeds, extending the evidence to discontinuous spiking dynamics.

The prediction algorithm was also evaluated across these systems, with task-specific adaptations of the directional response computation. Table 2 reports immediate post-update target-boundary classification.

The predictor achieved high four-way recall in five systems. In the hard-spiking SNN, discontinuous activation thresholds limited local derivative-based prediction, yielding 70.74% macro recall despite 95.31% accuracy. Together, the experiments demonstrate cross-system recurrence of the core TLI relations and identify architecture-dependent differences in their quantitative responses and predictive approximations. Figure 3 summarizes the six within-system TLI experiment chains.

Reinforcement: From TLI Open-Loop to TLIR Closed-Loop Dynamics

Theory-Derived Prediction of Feedback Dynamics

The established TLI relations show that training persistently reorganizes distributed functional support, while inference recruits and combines this learned support to produce behavior. When the consequences of inference are repeatedly fed back into subsequent training, the resulting updates can further reorganize the support on which future inference depends.

This led to a prediction: sustained, narrowly concentrated correct positive feedback could progressively favor the reinforced capability while weakening other already mastered capabilities. The predicted effect arises from feedback-directed functional reorganization, rather than incorrect rewards.

Experimental Tests and Results

RL-E05/E06 tested this prediction in a seventh system, a shared attention-free GRU policy. All four skills reached mastery before the experimental branches began. Every branch started from an identical parameter–AdamW state, and each complete two-decision episode received $r = 1$ only when correct, and $r = 0$ otherwise.

RL-E05 compared selective, balanced, and frozen feedback. RL-E06 further varied the allocation of 60 episodes per update over 3,200 updates (Table 3).

RL-E05 observed relative support concentration, cross-skill interference, and component-version-dependent trade-offs. Its stronger preregistered temporal-precedence criterion was not met.

In RL-E06, all 12 formal seeds exhibited dose-ordered changes. The reinforced skill's mean support share increased from 25.54% under balanced feedback to 35.20% under exclusive feedback, while the other skills' mean margins decreased from 9.474 to 3.962. Mild and high concentration already reduced these margins without reducing accuracy. Exclusive feedback decreased other-skill accuracy by 39.06 percentage points relative to balanced feedback.

Two recovery branches then started from the exact exclusive-feedback state at update 800.

Rebalanced feedback restored other-skill accuracy from 70.31% to 99.48% while preserving the reinforced skill at 100%. Training only the other three skills restored their accuracy to 99.83%, but reduced the previously reinforced skill to 89.06%.

These experiments establish that, in the tested shared system, the distribution of correct feedback can redirect persistent functional support, alter previously mastered capabilities, and reverse these changes through feedback redistribution.

TLIR Closed-Loop Dynamics

Reinforcement connects the consequences of inference to subsequent training actions. The resulting training updates reorganize learned support, which in turn changes future inference and its consequences. This feedback closure transforms the experimentally established TLI open-loop dynamics into the unified Training–Learning–Inference–Reinforcement (TLIR) closed-loop dynamics.

Unified Explanations of Positive and Negative Findings

Question and discovery order. A separate comparison of 21 historical positive/negative pairs asks whether common receiver–action–response–support–boundary–feedback relations explain both the success of local accounts and the findings that limit their scope. The examples below illustrate this shared interpretation.

Kernel dynamics versus feature learning. In the corresponding infinite-width lazy regime, NTK dynamics give a rigorous local training description (*1*); feature-learning parameterizations admit different limiting behavior (*21*). TLI locates both within receiver- and parameterization-dependent action–response regimes: a valid frozen-kernel projection does not imply that all finite-amplitude training trajectories preserve their features. The two results occupy distinct training regimes.

Sharp minima versus useful sharpness-aware training. Function-preserving reparameterizations can change raw parameter sharpness without changing generalization (*22*). Nevertheless, SAM improves performance in specified tasks by optimizing a neighborhood-robust loss (*3*). The paired explanation is that raw parameter geometry is not the complete learned functional state, while a sharpness-aware intervention can still change the *actual training action* beneficially. Thus a non-invariant explanatory scalar and an effective local control method can coexist.

Compression as a useful mechanism, not a universal requirement. General compression-stage claims fail in some architectures and measurement regimes (*23*), whereas appropriately controlling information bottlenecks can yield valid generalization guarantees (*24*). Persistent functional reorganization in TLI does not require a monotonic decrease of $I(X;T)$, but compression can be one regime-specific outcome of the receiving state, noise, and objective. This predicts *bounded utility without universal necessity*.

Delayed emergence and continued learning after mastery. In grokking, test capability can appear abruptly after training performance has saturated, although internal computational structure evolves in identifiable stages (*9*). Neural-collapse results likewise document continuing feature geometry after training error vanishes (*6*). TLI separates persistent support formation from the discrete target readout boundary. Continued internal formation and sudden observable transitions are thus compatible outcomes of the same measured distinction.

Forgetting versus preservation and recovery. Continual learning can damage older capabilities, while methods such as EWC protect selected high-importance parameter directions (*12*). In TLI, the signed response of an old target depends on overlap between the new action and its function-sensitive support, relative to remaining boundary slack. The GRU feedback forks supply a matched intervention example: concentrated *correct* feedback erodes other mastered skills, whereas rebalanced feedback largely restores them.

Beneficial self-training versus recursive collapse. Pseudo-label-based self-training improves accuracy in suitable data mixtures (*19*); recursively replacing original data with model-generated samples can instead erase minority tails (*20*), while accumulating real data can prevent that degradation in studied regimes (*25*). Both follow a feedback-data distinction: an inference-generated sample is neither intrinsically beneficial nor intrinsically harmful; its retained information and subsequent training action matter. For an under-retained mode with retention factor $0 < \rho < 1$, a conditional illustrative recursion is

$$q_{t+1} = (1-\alpha)\rho q_t + \alpha q_{\text{real}}. \tag{10}$$

At $\alpha = 0$ the mode contracts, whereas $\alpha > 0$ supplies a positive fixed-point floor if $q_{\text{real}} > 0$. The recursion models the case of a repeatedly under-retained data mode.

Theory-first extrapolation. Beyond historical comparisons, the TLIR relations have been used to derive conditional predictions and their test conditions before searching the relevant litera-

ture. Examples include post-capability max-margin selection under exponential-tail losses (*5*) and evolving-state stability at the Edge of Stability (*2*). These comparisons identify specific state, action, and boundary conditions under which local mathematical results arise. Future tests can directly intervene on these conditions and evaluate the predicted transitions.

Conclusion

Through systematic experiments across neural systems, this work reveals dynamical relations spanning neural-network training, learning, inference, and reinforcement. From capability formation and use to loss and recovery, TLIR establishes experimentally testable causal connections between observable performance changes and the organization and evolution of internal functional support. Neural-network capability evolution thus moves beyond a black-box performance phenomenon and becomes a dynamical process that can be investigated through precise experimental interventions and quantitative prediction.

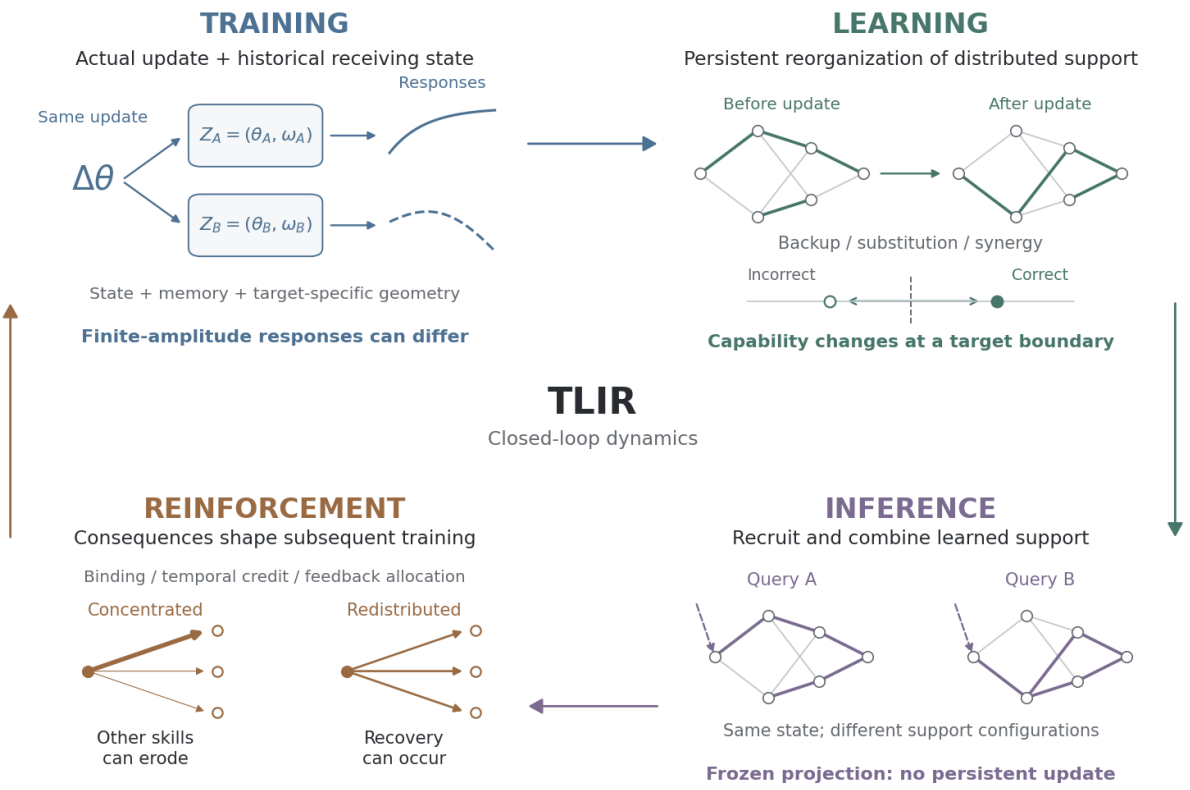


Fig. 1. TLIR closed-loop dynamics. Training, Learning, and Inference establish the open-loop core; Reinforcement feeds behavioral consequences into subsequent training.

Table 1. Seven neural systems: six TLI chains and a GRU feedback study.

System (optimizer)	Task / registered components
nanoGPT (Adam)	Cyclic next-token; 4 attention/MLP branches
ResNet-18 (SGD momentum)	CIFAR-100 classification; 4 residual stages
Diffusion U-Net (AdamW)	CIFAR-10 denoising; 4 U-Net paths
Vision-MoE (AdamW)	CIFAR-100 classification; 4 routed experts
GNN (RMSprop)	Graph reachability; 4 message-passing blocks
Spiking SNN (Adagrad)	8×8 digits; 4 spiking blocks
GRU (RL-E05/E06; AdamW)	Four-skill feedback; 4 shared hidden groups



Fig. 2. Selected evidence for Training, Learning, and frozen Inference in nanoGPT, and feedback-driven capability change and recovery in the shared GRU. (a1) Paused evolution delays capability formation; (a2) identical updates elicit different receiver responses; (a3) finite-amplitude response types; (a4) boundary, update, and receiver factors. (b1) Joint component removal crosses a target boundary; (b2) a realized update reallocates support; (b3) pre- and post-update target margins. (c1) Query-conditioned component effects; (c2) non-additive pair effects; (c3) learned-version rollback and exact restoration. (d1) Feedback-dose support redistribution; (d2) margin erosion before accuracy loss; (d3) rollback trade-offs; (d4) feedback rebalancing and recovery.

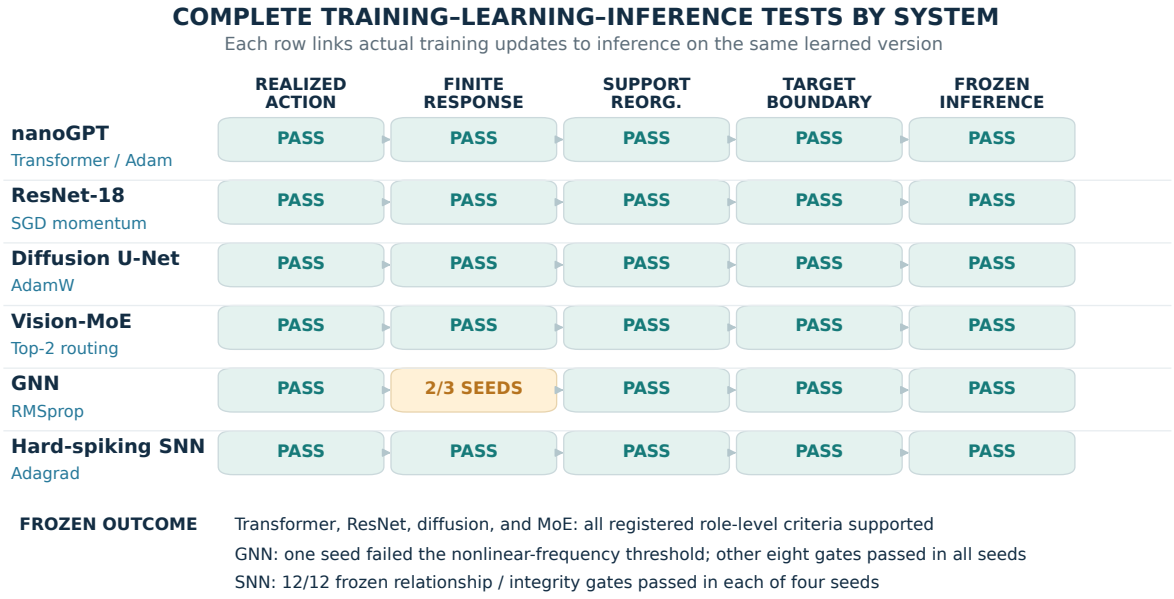


Fig. 3. Within-system training–learning–inference experiment chains. One GNN seed meets eight of nine registered gates, with nonlinear-response incidence below its specified threshold; SNN passes all twelve registered gates in four seeds.

Table 2. Post-update target-boundary classification reports. Macro recall averages four observed transition categories; high accuracy alone is not sufficient. Predictor details and evaluation sets differ by system.

System	Accuracy (%)	Four-way recall (%)
nanoGPT	91.43	91.49
ResNet-18	99.28	97.99
Diffusion U-Net	99.60	97.98
Vision-MoE	99.63	96.95
GNN[a]	98.54	98.62
Hard-spiking SNN[a]	95.31	70.74

[a]Independent prediction tests use registered GNN/SNN training histories.

Table 3. RL-E06 feedback episodes per update, by capability.

Feedback	q_0	q_1	q_2	q_3
Balanced	15	15	15	15
Mild	30	10	10	10
High	45	5	5	5
Exclusive	60	0	0	0

References and Notes

1. A. Jacot, F. Gabriel, C. Hongler, Neural tangent kernel: Convergence and generalization in neural networks. *Adv. Neural Inf. Process. Syst.* **31** (2018).

2. J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, A. Talwalkar, Gradient descent on neural networks typically occurs at the edge of stability. *Proc. ICLR* (2021).

3. P. Foret, A. Kleiner, H. Mobahi, B. Neyshabur, Sharpness-aware minimization for efficiently improving generalization. *Proc. ICLR* (2021).

4. N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio, A. Courville, On the spectral bias of neural networks. *Proc. Mach. Learn. Res.* **97**, 5301–5310 (2019).

5. D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar, N. Srebro, The implicit bias of gradient descent on separable data. *J. Mach. Learn. Res.* **19**, 1–57 (2018).

6. V. Papyan, X. Y. Han, D. L. Donoho, Prevalence of neural collapse during the terminal phase of deep learning training. *Proc. Natl. Acad. Sci. U.S.A.* **117**, 24652–24663 (2020).

7. N. Tishby, N. Zaslavsky, Deep learning and the information bottleneck principle. *IEEE Information Theory Workshop*, arXiv:1503.02406 (2015).

8. A. Power, Y. Burda, H. Edwards, I. Babuschkin, V. Misra, Grokking: Generalization beyond overfitting on small algorithmic datasets. arXiv:2201.02177 (2022).

9. N. Nanda, L. Chan, T. Lieberum, J. Smith, J. Steinhardt, Progress measures for grokking via mechanistic interpretability. *Proc. ICLR* (2023).

10. M. Belkin, D. Hsu, S. Ma, S. Mandal, Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proc. Natl. Acad. Sci. U.S.A.* **116**, 15849–15854 (2019).

11. P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, I. Sutskever, Deep double descent: Where bigger models and more data hurt. *Proc. ICLR* (2020).

12. J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, R. Hadsell, Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci. U.S.A.* **114**, 3521–3526 (2017).

13. D. Lopez-Paz, M. A. Ranzato, Gradient episodic memory for continual learning. *Adv. Neural Inf. Process. Syst.* **30**, 6467–6476 (2017).

14. M. Farajtabar, N. Azizan, A. Mott, A. Li, Orthogonal gradient descent for continual learning. *Proc. Mach. Learn. Res.* **108**, 3762–3773 (2020).

15. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30**, 5998–6008 (2017).

16. T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **33** (2020).

17. J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **35** (2022).

18. L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback. *Adv. Neural Inf. Process. Syst.* **35** (2022).

19. Q. Xie, M.-T. Luong, E. Hovy, Q. V. Le, Self-training with noisy student improves ImageNet classification. *Proc. CVPR*, 10687–10698 (2020).

20. I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, Y. Gal, AI models collapse when trained on recursively generated data. *Nature* **631**, 755–759 (2024).

21. G. Yang, E. J. Hu, Tensor programs IV: Feature learning in infinite-width neural networks. *Proc. Mach. Learn. Res.* **139**, 11727–11737 (2021).

22. L. Dinh, R. Pascanu, S. Bengio, Y. Bengio, Sharp minima can generalize for deep nets. *Proc. ICML*, 1019–1028 (2017).

23. A. M. Saxe, Y. Bansal, J. Dapello, M. Advani, A. Kolchinsky, B. D. Tracey, D. D. Cox, On the information bottleneck theory of deep learning. *Proc. ICLR* (2018).

24. K. Kawaguchi, Z. Deng, X. Ji, J. Huang, How does information bottleneck help deep learning?. *Proc. Mach. Learn. Res.* **202**, 16049–16096 (2023).

25. M. Gerstgrasser, R. Schaeffer, A. Dey, R. Rafailov, H. Sleight, J. Hughes, T. Korbak, R. Agrawal, D. Pai, A. Gromov, D. A. Roberts, D. Yang, D. L. Donoho, S. Koyejo, Is model collapse inevitable? Breaking the curse of recursion by accumulating real and synthetic data. *ICML Workshop on Foundation Models in the Wild* (2024).

Acknowledgments

Funding: This research received no funding.

Author contributions: Mian Wang conceived the study; developed the atomic generation-fact model, the Generation-Fact Graph, the GFG-based recursive scientific process, and the unified theory of training, learning, inference, and reinforcement; defined the scientific questions, research boundaries, and decision criteria; introduced the principal theoretical hypotheses and conceptual transitions; directed the research program; analyzed and interpreted validated results; developed prospective predictions and theoretical deductions; prepared the figures; and wrote and revised the manuscript.

Competing interests: There are no competing interests to declare.

Data, code and materials availability: The study's public evidence repository is organized by the scientific claims of this paper to support rapid machine-assisted auditing (https://github.com/wind342/tlir-research-evidence/tree/codex/tlir-paper-evidence-navigation-v1). A paper-to-evidence map connects the main claims, experiments, figure panels, and reported results to frozen protocols, source code, result records, provenance and checksum manifests, and independent verification procedures. The public evidence matrix specifies which results can be independently recomputed and which underlying checkpoints or runtime artifacts are not included in the public release.

Use of generative artificial intelligence: General-purpose AI systems, including OpenAI Codex and ChatGPT, served as scientific agents within the GFG-based recursive scientific process. Under the author's direction, these AI systems invented and implemented the four experimental instruments—CSRG-4C, realized-update causal forks, finite-amplitude update paths, and the identity-aligned target-boundary ledger—and designed, implemented, initiated, monitored, and executed the reported experiments. They performed analyses, causal interventions, matched replays, counterexample searches, and independent validation procedures, and contributed to manuscript drafting, editing, and LaTeX formatting. The reported numerical results were produced by the documented native programs and execution environments, not generated directly as text by AI models. The author defined the scientific questions, research boundaries, and decision criteria, reviewed the validated evidence, and accepts responsibility for the scientific claims and results.